QueryAPIGate HOW-TO GUIDE 42 · OPERATE

Run several instances behind a load balancer

Current as of QueryAPIGate **v0.49.0** · PDF made 4 November 2026

Latest version: anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?g=42-run-several-instances

Time: 30 minutes. **You'll end up with:** three QueryAPIGate instances serving as one - with Docker Compose or Kubernetes - that share keys, saved queries, rate limits and the cache, keep serving when one goes down, and can be upgraded one at a time.

Both were run end to end before this guide was written: the Compose deployment on one machine, and the Kubernetes manifests on a local cluster (kind), each with a real PostgreSQL store, Redis and traffic.

What several instances need

	WHY
A PostgreSQL store (`QUERYAPIGATE_DATABASE_URL`)	connections, saved queries and versions, keys, roles, history and the audit log, shared - SQLite volumes per instance would make separate servers
Redis (`QUERYAPIGATE_REDIS_URL`)	rate limits and the response cache, shared - without it each instance counts and caches on its own, and an alert says so
`queryapigate events` for live events	it reads every instance's runs from the store; route `/events` to it
The same secrets everywhere	`QUERYAPIGATE_API_KEY`, `QUERYAPIGATE_SECRET_KEY` and any `${VAR}` a connection uses
One gunicorn worker per container	the image's default - scale with containers, not workers, so each `/metrics` is one process

Option A: Docker Compose

The reference deployment is in the repository, under `deploy/scale-out/`: three instances, PostgreSQL, Redis, the events server and Caddy in front.

```shell
cd deploy/scale-out
cp .env.example .env && chmod 600 .env    # pin QUERYAPIGATE_IMAGE; set the key, passwords and SITE_ADDRESS
docker compose up -d
docker compose ps                         # six services, healthy
curl https://api.example.com/health
```

Caddy round-robins requests across the three instances, checks each one's `/health`, takes a failing one out of the rotation, and retries a request on another instance if one can't be reached. `/events` goes to the events server, and `/metrics` is blocked at the proxy (scrape the instances directly).

Option B: Kubernetes

`deploy/kubernetes/` has a Deployment of three replicas, the events server, Services, a PodDisruptionBudget and an Ingress (ingress-nginx; adapt the annotations for another controller):

```shell
cp secret.example.yaml secret.yaml        # fill in - or create the Secret from your secret manager
kubectl apply -f secret.yaml -f queryapigate.yaml
kubectl rollout status deploy/queryapigate
```

Use a managed PostgreSQL and Redis in production; `dev-dependencies.yaml` starts throwaway ones for trying it.

Settings in the manifest that matter, each found by running it:

- **`enableServiceLinks: false`.** Kubernetes puts `<SERVICE>_PORT=tcp://...` into every pod for each Service, so a Service named `queryapigate-events` sets `QUERYAPIGATE_EVENTS_PORT` - one of QueryAPIGate's own settings. The first deployment of these manifests crashed on exactly that. QueryAPIGate now ignores such a value with a warning naming the cause, and the manifest switches service links off.
- **A `preStop` pause and a 45-second grace period.** Without them, a rolling update dropped requests that arrived while a pod was being removed - 2 of 1,633 in the test. With them: 1 of 1,805 in one run (a timeout) and 0 of 1,896 in another.
- **`maxUnavailable: 0`, `maxSurge: 1`.** Replace one pod at a time - the previous release keeps serving on the upgraded store meanwhile.
- **Readiness and liveness probes on `/health`.** A pod gets traffic only once it answers.

Check that they act as one

```shell
curl https://api.example.com/api/v1/instances -H 'X-API-Key: ...'
```

```json
{"items": [{"role": "events", ...}, {"role": "serve", "host": "8e7a77dc4f96", "shared_limits": true,
...},
           {"role": "serve", "host": "937cabe5b957", "shared_limits": true, ...},
           {"role": "serve", "host": "db7a2881750c", "shared_limits": true, ...}],
 "problems": []}
```

Three `serve` instances and the events server, all with `shared_limits: true`, and no `problems` - on both. What else was verified:

CHECK	COMPOSE	KUBERNETES
Calls through the load balancer	answered by the three instances in turn	-
A key with `rate_limit: "4/minute"`, called across instances	200 ×4, then 429	200 ×4, then 429 (one call per pod)
A query against PostgreSQL	`[{"n": 100, "total": 7575.0}]`	the same
`/events`	runs arrived through the proxy	the events server healthy
One instance stopped, twelve calls	all twelve 200	-
A rolling update under steady traffic	-	0 of 1,896 failed (1 of 1,805 in another run)
`/metrics` through the proxy	403	-

An instance that stopped stays in `GET /api/v1/instances` for up to 90 seconds; a restarted one appears with a new id.

Day two

- **Upgrades:** one instance at a time - see Upgrade safely and DEPLOYMENT.md's Rolling upgrades. `GET /api/v1/instances` shows who is on which version meanwhile.
- **Monitoring:** scrape every instance's `/metrics`; the bundled Grafana dashboard sums them (guide 35).
- **Backups:** the store is PostgreSQL now - guide 33. Redis needs none.
- **Database connections:** each instance holds about nine connections to the store; see DEPLOYMENT.md's Shared metadata store.
- **Alerts** to know: `instances_not_shared` (some instances without Redis), `instances_versions_differ` (a rolling upgrade not finished), `rate_limits_not_shared` (Redis unreachable - limits fall back to per instance).

Next steps

- Put QueryAPIGate behind a reverse proxy - TLS and forwarded addresses.
- Rate-limit a key, or the whole server.
- Scaling out.